

- 人工智能实验报告 第4周
 - 一.实验题目：一阶逻辑归结推理
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 3.创新点&优化(如果有)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）
 - 四.参考资料(可选)

人工智能实验报告 第4周

姓名:陈安康 学号:22320021

一.实验题目：一阶逻辑归结推理

二.实验内容

1.算法原理

一阶逻辑归结推理是一种逻辑归结原理的自动推理方法，用于判断逻辑命题的真假或者解决逻辑问题。

一般的子句归结原理简述如下：假设有两个一般子句（文字的析取） f_1 与 f_2 位于同一子句集（子句的合取），其中 p ， c_1 与 c_2 表示一般文字，子句 f_1 与 f_2 分别包含 p 文字及其否定：

$$\begin{aligned}f_1 &= \neg p \vee c_1 \\f_2 &= p \vee c_2\end{aligned}$$

则运用归结原理，可以消去文字 p ，将 f_1 与 f_2 子句归结成子句 f_3 ：

$$f_3 = c_1 \vee c_2$$

通过不断地归结，如果生成了一个空子句，则说明原始逻辑公式是不可满足的。子句归结方法通常与反证法结合，即将目标公式的否定化成文字析取的形式加入子句集，通过子句归结得到空子句来说明原始逻辑

公式的不可满足性，从而说明推理的正确性。

一阶逻辑归结推理在两个子句进行归结时，需要先进行两个逻辑项的最一般化替换，即通过MGU算法将其中的子句中不一致的项替换成一致的，再进行归结得到新子句，直到得到空子句。

MGU算法原理简述如下：

从两个谓词相同的子句中每一次找出不匹配项，如果其中一项是变量，另一项是不含该变量的项，则将该替换规则加入最一般合一，同时将该替换应用到两个子句中，继续进行这个过程，直到子句中的项完全一致，否则合一失败。

代码实现上主要分为两大部分：用于MGU实现的代码与实现子句归结的代码。首先介绍MGU代码实现思路：

考虑到谓词相同的项不仅含有单一变量和常量，还含有函数，这涉及到括号拆分的过程，对于MGU核心函数unity的实现（该函数实现判断两项之间能否进行归结并在可以归结时找出最一般替换）上，我采用了**分情况考虑以及递归**的思路，将传入的项的情况进行如下划分，分别采用不同策略进行解决，细节如下：

1.传入的项均为单一变量与常量，不嵌套与函数。则进行判断是否为两个常量，若是，则判断是否相等，不相等则返回否定，表示合一失败。相等则返回成功，但不加入最一般合一集合中。

若有一项为变量，则在最一般合一集合中查找其是否有替换值，若有，则递归返回其替换值与另一项的合一结果。若没有，则最一般合一添加该变量替换为另一项常量的替换规则，并返回成功。

若均为变量，在最一般合一集合中查找是否已经存在替换值，递归返回替换值的unity

2.传入参数有一项是函数，则将函数拆分，得到对应参数项，从合一集中查找每一项的替换是否存在，存在就进行替换，再将函数组合后将该组替换加入合一集。其中若里面的参数仍有函数，则巧妙的递归利用unity求解替换项，并从合一集删除递归来的最新替换来保证合一集正确性。

3.传入参数是两个函数，则首先判断函数名称是否一致，不一致则返回合一失败。否则递归的对每一项进行求解，返回每次求解结果的合取。

伪代码如下：

```
unity(f1,f2) :  
    if f1与f2均为单一变量或者常量:  
        if f1 与 f2 均为常量 :  
            if f1 = f2 :  
                return True  
            else :
```

```

        return False 合一失败
    if f1 f2 有一项为变量, 假设为f1 :
        查找 f1 在 最一般合一集合中是否存在替换, 记为val
        if val 存在 :
            return unity(val,f2)
        else :
            合一集合添加[f1 : f2] return True
    if f1 f2 存在一项为函数式, 假设为 f2 :
        f2进行括号拆分, 分离出参数集合记为items
        遍历参数集合里面的元素, 记为it:
            if it与f1相同 :
                return False 合一失败
            if it不为函数 :
                查找合一集合, 已经有替换的进行替换, 加入到items中
            如果it为函数 :
                unity(f1,it),如果成功则令得到的最新替换替换it, 将合一集
最新加入的替换删除
        合并items为f2',合一集添加{f1:f2'} return True
    if f1 f2 均为函数 :
        判断函数类型是否相同, 不一致则返回错误, 合一失败。否则拆分f1 f2得到参数集
合分别为items1, items2
        遍历两个集合, 假设两集合对应项为it1 it2 :
            flag = unity(it1, it2)
            if flag = False :
                return False
        return True

```

在主体函数MGU的实现上, 对两个子句的项进行对应逐一比对, 如果不一致则进行合一, 直到达到项完全相同或者合一失败。

为保证结果正确性, 还需要**对差异项的合一顺序进行规定**, 这里要求首先对单一变量和常量的情况进行优先归结, 其次归结函数与函数的情况, 最后归结单一变量与函数的情况, 这样的顺序可以保证在归结到对应情况是, 应该存在的替换关系是存在的。

实现上考虑采用队列存储后两种情况的项, 然后依次处理。从而达到按照规定顺序进行处理项的目的。

MGU算法的主体思路大致如上。实现在MGU函数和unity函数中。接下来介绍子句归结的算法实现思路。

子句归结部分主体思路是双重循环枚举子句集中的每一个位合一过的子句, 尝试进行合一和归结, 将归结产生的新子句加入子句集, 继续进行尝试, 直到找出空子句 (这里假定都是能推出答案的)。

为了防止重复归结, 设置visited数组记录已经归结过的子句编号。并且这个visited数组还可以复用作查找子句归结依赖关系的映射, 即当前子句是由哪两个子句归结而来。

在归结过程中, 为了简化归结次数, 会在每次归结完成后对子句进行排序, **优先遍历单字句**。因为空子句是由单字句最终归结得到的, 这样可以减少归结产生无用子句的次数, 以下两个图是采用单字句优先与不采用单字句优先遍历的归结步骤图:

```

> 91: ('~L(john,snow)', '~S(john)')
> 92: ('~A(mike)',)
> 93: ('~L(mike,rain)', '~A(mike)')
> 94: ()
len(): 95

```

```

> 098: ('C(tony)', '~A(mike)', 'C(mike)')
> 099: ('C(tony)', '~A(mike)', '~C(mike)')
> more: [100:128]
len(): 128

```

可以看到归结得到答案所需的子句数目大幅减少。

尽管归结产生的无用子句变少，但其数目仍然很多。为了呈现步骤的效果，我们需要从空子句**回溯找出真正有用的子句加入归结步骤**中。实现思路是凭借子句与visited中的子句编号——对应的关系，可以从**空子句出发借助visited数组回溯得到所有有用的子句**，然后进行步骤输出。这段代码位于backtracking函数与print_steps函数中。

2.关键代码展示(可选)

回溯代码展示如下：

```

def backtracking(visited,tmp2_kb,kb,pos_in_ziju) :
    #visited是按顺序存放每个子句的两个父子句编号的数组（不包括初始子句的），tmp2_kb是按照归结
    #顺序产生的所有的子句（包括初始子句），kb存放初始子句，后面那个是存的abc这个信息，多输入了，
    #这里没有用
    for i in range(len(kb)) :
        visited.insert(0,[])#扩充visited子句让visited长度与tmp2_kb一致，方便后面——对
        应
    res = []
    step = len(kb)
    #visited和tmp_kb里面的每个归结产生的子句是--对应的
    i = 0
    step_index = [len(tmp2_kb)-1]
    for it in step_index :
        pos = visited[it]
        for e in pos :
            if e > step - 1 :
                step_index.append(e)
    step_index = list(set(step_index))#去重
    step_index = sorted(step_index)#对子句在tmp_kb里面的标号从小到大排序
    for i in step_index :#step_index里面存放了所有有用的非原始的子句的标号
        res.append(tmp2_kb[i])
    #step_index和visited同时也天然形成了给了我们查询对应关系的一个映射表
    return res,step_index

```

MGU代码展示如下：

```
def MGU(str1,str2) :
    ans = {}
    #这几个队列就是存储还未归结的项，调整归结顺序的
    #judge_var是判读项是不是变量的一个函数
    #项是不是函数通过判断有没有"("来判断
    #split_items函数是用来拆分括号获得每一项的，此函数中还有一个递归调用防止误拆项为多参数
    函数中的括号
    queue1 = {}
    queue2 = {}
    queue3 = {}

    _,items1 = split_items(str1)
    _,items2 = split_items(str2)
    if len(items1) != len(items2) :
        return {}
    n = len(items1)
    for i in range(n) :
        if items1[i] != items2[i] :
            if ((judge_var(items1[i]) and (not judge_var(items2[i]) and '(' not in
items2[i])) or ((not judge_var(items1[i]) and '(' not in items1[i]) and
judge_var(items2[i]))) :
                flag = unity(items1[i],items2[i],ans)
                if not flag :
                    return {}
            elif judge_var(items1[i]) and judge_var(items2[i]) :
                queue3[items1[i]] = items2[i]
            elif '(' in items1[i] and '(' in items2[i] :
                queue1[items1[i]] = items2[i]
            elif '(' in items1[i] or '(' in items2[i] :
                queue2[items1[i]] = items2[i]
            else :
                flag = unity(items1[i],items2[i],ans)
                if not flag :
                    return {}
    for key,value in queue3.items() :
        flag = unity(key,value,ans)
        if not flag :
            return {}
    for key,value in queue1.items() :
        flag = unity(key,value,ans)
        if not flag :
            return {}
    for key,value in queue2.items() :
        flag = unity(key,value,ans)
        if not flag :
            return {}
    return ans
```

3.创新点&优化(如果有)

采用回溯方法筛选出有用子句，简化步骤表达。

三.实验结果及分析

1.实验结果展示示例（可截图可表可文字，尽量可视化）

事例运行结果如下：

```
PS C:\Users\86135\Desktop\python> & C:/ProgramData/anaconda3/python.exe c:/Users/86135/Desktop/python/final_week2.py
1 ('GradStudent(sue)',),
2 (~GradStudent(xx)', 'Student(xx)'),
3 (~Student(xx)', 'HardWorker(xx)'),
4 (~HardWorker(sue)',),
5 R[1,2a]{xx=sue} = ('Student(sue)',),
6 R[3a,4]{xx=sue} = (~Student(sue)'),),
7 R[5,6]{} = ()
PS C:\Users\86135\Desktop\python>
```

作业1运行结果：

```
16 R[14,15]{} = ()
PS C:\Users\86135\Desktop\python> & C:/ProgramData/anaconda3/python.exe c:/Users/86135/Desktop/python/final_week2.py
1 ('A(tony)',),
2 ('A(mike)',),
3 ('A(john)',),
4 ('L(tony,rain)',),
5 ('L(tony,snow)',),
6 (~A(xx)', 'S(xx)', 'C(xx)'),
7 (~C(yy)', '~L(yy,rain)'),
8 ('L(zz,snow)', '~S(zz)'),
9 (~L(tony,uu)', '~L(mike,uu)'),
10 ('L(tony,vv)', 'L(mike,vv)'),
11 (~A(ww)', '~C(ww)', 'S(ww)'),
12 R[5,9a]{uu=snow} = (~L(mike,snow)'),),
13 R[6c,11b]{xx=ww} = (~A(ww)', 'S(ww)'),
14 R[2,13a]{ww=mike} = ('S(mike)',),
15 R[8a,12]{zz=mike} = (~S(mike)'),),
16 R[14,15]{} = ()
PS C:\Users\86135\Desktop\python>
```

作业2运行结果：

```

● PS C:\Users\86135\Desktop\python> & C:/ProgramData/anaconda3/python.exe c:/Users/86135/
1 ('On(tony,mike)',),
2 ('On(mike,john)',),
3 ('Green(tony)',),
4 ('~Green(john)',),
5 ('~On(xx,yy)', '~Green(xx)', 'Green(yy)'),
6 R[3,5b]{xx=tony} = ('Green(yy)', '~On(tony,yy)'),
7 R[4,5c]{yy=john} = ('~Green(xx)', '~On(xx,john)'),
8 R[1,6b]{yy=mike} = ('Green(mike)',),
9 R[2,7b]{xx=mike} = ('~Green(mike)',),
10 R[8,9]{} = ()
○ PS C:\Users\86135\Desktop\python>

```

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

MGU函数时间复杂度是 $O(k+l+z)$,其中 k 是检查的两个文字项的长度， l 是情况2的项对的个数， z 是情况3项对的个数。子句归结函数的时间复杂度比较难以估算，因为子句集是动态增长的，时间复杂度估计为 $O(N^2)$,其中 N 为初始子句的个数。

四.参考资料(可选)